Chap 16 - Maps/Multimaps

Maps are template container objects that store data in a key/value pair. The key is used for retrieval of the value.

There two main categories of maps: ordered and unordered. Ordered maps are internally sorted and some form of binary search tree is usually used to implement them (Red/Black trees are normally used but AVL and Splay trees may as well). Searching an ordered map has a time complexity of O(log(n)).

Unordered maps are generally implemented using a hash table so searching them has a time complexity of O(1).

The key is unique in either type of map. If you need to have multiple key values you should use a multimap which also can be ordered or unordered.

1. Create a map called "pop" that takes a key/value pair of string and unsigned int. Look up online the populations of the countries: China, India, USA and Tokelau. Using an *initializer* list, initialize the pop map object with the country name and its population.
    a) Using a ranged for loop print out the key/value pairs of pop. How is it sorted?
    b) Using *operator[]*() print out the population of Tokelau
    c) Using *operator[]*() print out the population of Japan (even though Japan is not in the map.
        - What does it return?
        - Has the map changed? How many elements does it have now?
        - Using the map *find*() method find the Japan entry that operator[] added in 1.c. Change its value to the right population.

    d) Using the *operator[]*() method add the population of Argentina to pop
    e) Using the *at()* method print out the population of India
    f) Using the *at()* method print out the population of Brazil (even though it is not there)
        - What happens?
        - How should you properly use at to catch when it attempts to retrieve a value that is not there?
        - How does operator[] and the at() method differ?
    g) Using operator[]() add the Population of Mars to pop (pretend Mars has a population of 42). Then erase the Mars entry from pop.
    h) Using insert add in the population of Russia
    i) Using insert add in the populations of Ghana, Myanmar and Spain (this can be done in one line of code
    j) Using emplace add in the population of France.
    k) Create a new map with a key/value pair of string and unsigned int called rpop. Have rpop perform a reverse sort on the key(hint: see the constructor for maps).
        - Using rpop's insert method, insert the elements of pop into rpop (can be done with one line of code)
        - Print out rpop's values and confirm the order is reverse.

l) Delete all the countries from Myamar to the USA (that should be Myamar, Russia, Spain, Tokelau, and the USA). Use the lower_bound(), upper_bound() and erase() methods()

2. Prompt the user to enter text. Create a map called word_count with key/value pair of string and unsigned int. Keep a word count for each word in the word_count map.

   For example if the user entered the text: "The dog and the cat ran out of the barn" then word_count["the"] would equal 3 and word_count["dog"] would equal 1.

   a) Print out each word and its word count.
   b) Print out each word and its word count BUT sorted descending by word count (hint: there are a number of ways to perform this but think back to the Algorithms exercise and look at what functions can help)

3. In order for a class to be a key type for a map, there needs to be a way to perform comparisons.
   a) Create a class called employee that contains a first and last name like below

```
class employee {
private:
        string firstname;
        string lastname;
public:
        employee(const string& f, const string& l) :firstname(f), lastname(l) {}
        employee() = delete;  //  no default constructor...
        // maybe add more?
};
```

   b) Create a map called 'Salary' that contains a key/value pair of an employee object and a double. Since employee is the key field there needs to be a way to compare two employee objects for sorting. Two ways to do this… either create comparison operators for the employee class (operator<()) or pass a compare functor as a third template argument.

      Note: to sort employees sort by the lastname unless they are equal, if so sort by the first name

Multimaps

A Multimap is similar to a Map except it can contain duplicate keys. Because of this, Multimaps do not have the operator[] or the at() method. To get values from the duplicate keys you need to use the equal_range() method.

4. Create the following Multimap of test scores

```
multimap<string, int> test;
test.insert({
        {"Ken",42},
        {"Dhruva",21},
        {"Darshan",31},
        {"Ken", 19},
        {"Dhruva",10},
        {"Darshan",5},
        {"Ken", 5} }
);
```

   a) Print out all the score belonging to Ken
   b) Create a vector that contains all the unique keys of test
      (Note1: you can do this in one/two lines of code… look at algorithm functions)
      (Note2: this would a perfect example for using sets… maybe next homework assignment 😉)
   c) Sum up all the scores for each in individual. Your printout should look like this:

```
Darshan 36
Dhruva  31
Ken     66
```

   d) Print out all the names whose score is greater than 20

5. Perform the following
   a) Write a **template** function that looks like:

```
template <typename T>
void bucket_stats(T& umap){...}
```

      Umap will be a unordered_map and it should print out the following statistics:
      - Size
      - Number of buckets
      - Load factor
      - Max load factor
   b) Create an empty unordered map called **umap** with a key /value pair of int/int

c) Make a for loop that loops from *i=0* while *i<= umap*'s bucket count (please put the count into a temporary variable).
d) In the for loop insert the into umap the *key = i* with an arbitrary value (could also be *i*)
e) After each insert call *bucket_stats(umap)*
f) Notice the *load factor* as we keep inserting values. What is it displaying?
g) What happens when we go over the number of buckets? This happens because we have a default max load factor of 1.0.
h) Change the code so that after creating umap you set it's max load factor to 0.5 and re-run the code. When does the number of buckets increase?

   Generally speaking setting a max_load_factor = 1.0 is not optimal. Normally a value of 0.7 or 0.8 works best.

6. Rehash vs Reserve
   a) Create an empty unordered map called **umap** with a key /value pair of int/int
   b) Set its max load factor = 0.5
   c) Call *umap.rehash(1024)* and then call your bucket_stats function passing umap. Note the number of buckets
   d) Comment out the line *umap.rehash(1024)* and instead call umap.reserve(1024). Run the code and look at the bucket count. What is the difference?
   Seeing this what is the difference between rehash() and reserve()?

7. Customized Hash for user defined keys – Most scalar types and a number of STL types in C++ have a hash function (technically it is functor) associated with it. For example:

```
auto h = hash<string>()(string("joe"));
h = hash<int>()(42);
```

Note: Since hash isn't a function but a functor we need to call it by forcing the operator() method

If the key of the unordered_map is a user defined object it must have a hash functor that takes the object type as an argument and an equivalence method (ie operator==()). If it takes multiple members of the class to make an object unique, then you need to create a hash functor that somehow applies a hash to all the fields that make it unique.

Say for example we have an object that represents a specific vaccine batch… the multiple fields would be: Manufacturer(string), id version(string), batch number (int)

   a) Create a class called vaccine that contains the string members manufacturer and id_version and an integer called batch_number. Add a constructor that assigns the three members. Also add an operator=() method.

b) Create a hash functor that implements the operator() method. It should look like:

```
struct vaccine_hash{
        std::size_t operator()(const vaccine & v) const {
                // do work here...
        }
};
```

- The return type is *size_t*
- In the operator() method get the hash values for the manufacturer, id_version, and batch_number
- Calculate and return the new hash value using a combination of the three hash values. A "decent" hash would be something like this:

  (hashmanufacturer ^ (hashid_version << 1)) >> 1 ^ (hashbatch_number << 1)

  Creating a good hash is not trivial… and there are libraries for doing so. A popular one is the hash_combine function from the Boost library.

c) Create an unordered_map that has a key/value pair of vaccine/int. Because the key(vaccine) is a user defined class you need to pass the hash functor (vaccine_hash) as the third template argument (ie. *unordered_map< vaccine, int, vaccine_hash > vac_map*).

d) Insert the following objects

```
vac_map.insert({
        {{"Pfizer","23ki21",1},890},
        {{"Pfizer","23ki21",2},721},
        {{"Pfizer","23ki21",3},652},
        {{"Pfizer","23ki23",1},1012},
        {{"Pfizer","23ki23",2},1618},
        {{"Pfizer","23ki23",4},813},
        {{"Moderna","oq112",40},900},
        {{"Moderna","oq112",50},600},
        {{"Moderna","oq224",10},300},
        {{"Moderna","oq224",20},400},
        {{"Moderna","oq224",80},800},
        {{"Moderna","oq330",30},700}
        });
```

e) Detecting bucket collisions- Normally the begin and end methods allows you to iterate through all value… but if you pass a number to begin(bucket_number) it will return the first value in that particular bucket.

For example to print out all the values in bucket 1 you can perform the following:

```
for (auto bucket_it = umap.begin(1); bucket_it != umap.end(1); ++bucket_it) {
        cout << " " << bucket_it->second; ;
}
```

Update your *bucket_stats()* function to print out the contents of all the buckets in the passed unordered_map.

Call bucket_stats and pass the vac_map object from problem #7. Are there any collisions?